第 8 讲：二叉搜索树、层序遍历与树的综合题

0. 本讲的核心问题

普通二叉树只规定每个节点最多有两个孩子，并不规定值的大小关系。

二叉搜索树 BST 进一步要求：

```
        <     <
```

这种全局有序性使查找可以像二分查找一样排除一整棵子树。

本讲还会学习树的 BFS：层序遍历。DFS 关心递归结构，BFS 关心同一深度的节点。

---

1. BST 的定义是全局约束

正确示例：

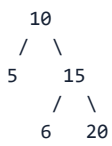


对于每个节点：

- 左子树所有值都比它小；
- 右子树所有值都比它大；
- 左右子树本身也必须是 BST。

注意“所有值”，不只是直接孩子。

下面不是 BST：



虽然 `6 < 15`，但 `6` 位于根 `10` 的右子树，却小于 `10`，违反全局约束。

重复值如何处理取决于题目定义。LeetCode 的验证 BST 通常要求严格不等，不允许重复值。

---

2. BST 搜索

```
[python]
def search_bst(root, target):
    current = root

    while current is not None:
        if target == current.val:
            return current

        if target < current.val:
            current = current.left
        else:
            current = current.right

    return None
```

每次比较后都排除一整棵子树。

复杂度取决于树高 `h`：

```
O(h)
O(1)  #
```

平衡树中 `h=O(log n)`；倾斜树中 `h=O(n)`。BST 不自动保证平衡。

递归版本：

```
[python]
def search_bst(root, target):
    if root is None or root.val == target:
        return root

    if target < root.val:
        return search_bst(root.left, target)

    return search_bst(root.right, target)
```

---

3. BST 插入

沿搜索路径找到空位置：

```
[python]
def insert_into_bst(root, value):
    if root is None:
        return TreeNode(value)

    if value < root.val:
        root.left = insert_into_bst(root.left, value)
    elif value > root.val:
        root.right = insert_into_bst(root.right, value)

    return root
```

这个版本忽略重复值。具体策略必须服从题目定义。

为什么需要：

```
[python]
root.left = insert_into_bst(root.left, value)
```

因为当左子树为空时，递归会创建并返回新节点，父节点必须把 `.left` 连接到它。

---

4. 中序遍历为什么有序

中序顺序是：

```
          →     →
```

BST 中：

```
             <     <
```

因此中序遍历得到严格递增序列。

```
[python]
def inorder_values(root):
    result = []

    def dfs(node):
        if node is None:
            return

        dfs(node.left)
        result.append(node.val)
        dfs(node.right)

    dfs(root)
    return result
```

这条性质可以解决验证 BST、第 K 小元素、寻找相邻差值等问题。

---

5. 验证 BST：错误方法与正确方法

5.1 只比较直接孩子是不够的

错误思路：

```
[python]
def only_checks_children(node):
    if node.left and node.left.val >= node.val:
        return False
    if node.right and node.right.val <= node.val:
        return False
    return True
```

它无法发现右子树深处出现一个小于祖先的值。

5.2 方法一：为每个节点维护允许区间

根允许范围是：

```
(-∞, +∞)
```

进入左子树时，上界变成当前值；进入右子树时，下界变成当前值。

```
[python]
def is_valid_bst(root):
    def validate(node, lower, upper):
        if node is None:
            return True

        if not (lower < node.val < upper):
            return False

        return (
            validate(node.left, lower, node.val)
            and validate(node.right, node.val, upper)
        )
```

```
        return validate(root, float("-inf"), float("inf"))
```

函数契约：

> `validate(node, lower, upper)` 判断整棵子树的所有值是否严格位于给定开区间。

5.3 方法二：检查中序序列严格递增

不必保存完整列表，只保存前一个访问值：

```
[python]
def is_valid_bst(root):
    previous = None

    def inorder(node):
        nonlocal previous

        if node is None:
            return True

        if not inorder(node.left):
            return False

        if previous is not None and node.val <= previous:
            return False

        previous = node.val

        return inorder(node.right)

    return inorder(root)
```

不要用某个普通数字作为初始 `previous`，否则可能与合法节点值冲突。使用 `None` 或负无穷。

---

6. 第 K 小元素

BST 中序序列递增，所以中序访问的第 `k` 个节点就是第 K 小。

```
[python]
def kth_smallest(root, k):
    stack = []
    current = root

    while stack or current is not None:
        while current is not None:
            stack.append(current)
            current = current.left

        current = stack.pop()
        k -= 1

        if k == 0:
            return current.val

        current = current.right
```

这个迭代中序可以在找到第 K 个元素时提前停止，不必遍历整棵树。

栈中保存“已经到达但尚未访问值的祖先节点”。

最坏时间 `O(n)`，额外空间 `O(h)`；如果 `k` 很小，可能提前结束。

---

7. BST 最近公共祖先

给定 BST 中两个节点 `p q`：

- 如果二者都小于当前值，公共祖先在左边；
- 如果二者都大于当前值，公共祖先在右边；
- 否则当前节点位于二者之间，或者等于其中之一，就是最近公共祖先。

```
[python]
def lowest_common_ancestor_bst(root, p, q):
    current = root

    while current is not None:
        if p.val < current.val and q.val < current.val:
            current = current.left
        elif p.val > current.val and q.val > current.val:
            current = current.right
        else:
            return current

    return None
```

时间 `O(h)`，额外空间 `O(1)`。

普通二叉树没有大小关系，需要不同方法。

---

8. 普通二叉树的最近公共祖先

函数契约：

> 返回当前子树中 `p q` 的最近公共祖先；如果只找到其中一个，则把找到的节点向上传递；如果都没找到，返回 `None`。

```
[python]
def lowest_common_ancestor(root, p, q):
    if root is None or root is p or root is q:
        return root

    left_result = lowest_common_ancestor(root.left, p, q)
    right_result = lowest_common_ancestor(root.right, p, q)

    if left_result is not None and right_result is not None:
        return root

    if left_result is not None:
        return left_result

    return right_result
```

如果左右子树各返回一个非空结果，说明 `p q` 分处两侧，当前节点就是最近公共祖先。

这道题再次体现：递归代码是否自然，取决于返回值含义是否定义清楚。

---

9. 层序遍历：树上的 BFS

目标：按层返回节点值。

```
    3
   / \
  9   20
```

```
    /  \
   15   7
```

输出：

```
[python]
[[3], [9, 20], [15, 7]]
```

使用队列：

```
[python]
from collections import deque

def level_order(root):
    if root is None:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        level = []

        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)

            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)

        result.append(level)

    return result
```

循环开始时，队列中的前 `level_size` 个节点恰好属于当前层。处理它们时加入的是下一层。

时间 `O(n)`，空间最坏 `O(w)`，其中 `w` 是树的最大宽度，最坏可达 `O(n)`。

---

10. 层序遍历的变形

10.1 右视图

从右侧看，每层只能看到最右节点：

```
[python]
from collections import deque

def right_side_view(root):
    if root is None:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)

        for index in range(level_size):
            node = queue.popleft()

            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)
```

```
                if index == level_size - 1:
                    result.append(node.val)

        return result
```

10.2 每层平均值

对本层累加并除以 `level_size`。

10.3 锯齿形层序遍历

仍然按正常 BFS 收集每层，只在输出时根据层号决定是否反转该层。不要为了输出方向破坏队列的正常扩展逻辑。

---

11. 判断完全二叉树的直觉

完全二叉树要求除最后一层外都填满，最后一层从左到右连续填充。

层序遍历中，一旦遇到空位置，之后不应再出现非空节点。

```
[python]
from collections import deque

def is_complete_tree(root):
    queue = deque([root])
    seen_none = False

    while queue:
        node = queue.popleft()

        if node is None:
            seen_none = True
            continue

        if seen_none:
            return False

        queue.append(node.left)
        queue.append(node.right)

    return True
```

这里故意把 `None` 也入队，用它表示树形结构中的空位置。

---

12. BST 删除的三种情况

删除节点比插入复杂。设要删除的节点为 `node`：

1. 没有子节点：直接删除；
2. 只有一个子节点：用唯一子节点替代它；
3. 有两个子节点：用右子树最小节点（中序后继）或左子树最大节点替代，再删除那个替代节点。

递归实现：

```
[python]
def delete_node(root, key):
    if root is None:
        return None

    if key < root.val:
        root.left = delete_node(root.left, key)
    elif key > root.val:
        root.right = delete_node(root.right, key)
```

```
        else:
            if root.left is None:
                return root.right
            if root.right is None:
                return root.left

            successor = root.right
            while successor.left is not None:
                successor = successor.left

            root.val = successor.val
            root.right = delete_node(root.right, successor.val)

        return root
```

这部分理解即可，初学阶段优先熟练搜索、验证和中序性质。

---

13. 怎样选择 DFS、BFS 或 BST 性质

问题	优先思路
最大深度、直径、路径	DFS 递归
按层输出、最小层数	BFS 队列
BST 查找或插入	比较大小，只走一侧
BST 第 K 小	中序遍历
BST 验证	上下界或中序递增
BST 最近公共祖先	利用大小关系
普通树最近公共祖先	后序递归

同一道题可能有多种方法。选择时看题目真正需要的是层、路径、子树结果还是有序性。

---

14. 复杂度中的平衡与倾斜

BST 搜索常被简单说成 `O(log n)`，但只有树较平衡时才成立。

平衡：

```
    4
   / \
  2   6
```

高度约为 `log n`。

倾斜：

```
1
 \
  2
   \
    3
```

高度为 `n`，搜索退化到 `O(n)`。

因此更准确的写法是：

```
O(h)
O(log n)
O(n)
```

15. 常见错误

15.1 把 BST 条件只检查到直接孩子

必须维护来自所有祖先的范围，或验证完整中序序列。

15.2 中序遍历的 `previous` 没有正确更新

比较当前节点后，应立即更新再进入右子树。

15.3 层序遍历动态使用 `len(queue)`

处理一层前保存 `level_size`，否则下一层节点会混入当前层。

15.4 忘记空树

层序遍历若直接 `deque([None])`，随后访问 `.val` 会出错，除非算法特意处理 `None`。

15.5 默认 BST 一定平衡

复杂度应先写成 `O(h)`。

15.6 普通树 LCA 和 BST LCA 混用

只有 BST 才能利用值的大小决定方向。

16. 本讲检查表

```
1.          BST
2.
3.
4.
5.
6.
7. BFS           level_size
8.
9.          O(h)    O(n)
10.
```

17. 本讲总结

1. BST 的约束作用于整个左右子树，不只是直接孩子。
2. BST 搜索、插入和删除复杂度是 `O(h)`，不一定总是 `O(log n)`。
3. 中序遍历 BST 得到严格递增序列。
4. 验证 BST 可使用祖先上下界或中序递增性质。
5. 第 K 小元素是中序遍历的第 K 个节点。
6. 层序遍历使用队列，并在每层开始前固定层大小。
7. 普通树和 BST 的最近公共祖先使用不同信息。